

40% 50% 10%

scala 2.12

Semaine 1 : Introduction Big Data et Map-Reduce

存储成本下降

网络性能上升

终端越来越多

用户越来越多

应用越来越多

每天产生越来越多的数据

o Ko Mo Go To Po Eo Zo Yo

每天产生 Eo 级别的数据

big data

3V 基本原理 fondamentaux:

Volume 容量: 处理和存储大量数据

traiter et stocker une grande masse de données

Vélocité 速度: 在短时间内生成大量数据

production massive de données en peu de temps

Variété 多样性: 各种格式的结构化/非结构化数据

données structurées/non structurées sur des formats variés

第 4V:

Véracité 真实性: 确保数据完整性 (过时、准确等)

assurer l'intégrité des données (obsolescence, justesse ...)

第 5V:

Valeur 价值: 以公允价值评估数据, 使其盈利

évaluer les données à leur juste valeur pour qu'elles soient rentables

Map-Reduce

MapReduce 是一种用于大规模数据集并行处理的编程模型，它将计算分为 Map 阶段的键值对映射和 Reduce 阶段的聚合汇总，支持在分布式系统中高效处理和生成数据。

MapReduce est un modèle de programmation utilisé pour le traitement parallèle de grands ensembles de données, qui divise le calcul en une phase Map pour l'association des paires clé-valeur et une phase Reduce pour l'agrégation, permettant de traiter et générer efficacement des données dans un système distribué.

la phase de map

以<键, 值>形式读取数据
处理
以<键, 值>形式输出数据

分片处理 (split)
调用 map 函数
生成中间结果

map 任务的数量等于输入数据被分割后的 split 数量，也就是说，每个 split 对应一个 map 任务。

Map : (K1,V1) → list(K2,V2)

La phase intermédiaire(shuffle)

在 maps 和 reduces 之间转移，分选，合并数据

maps 端:

本地存储 (stockage local): map 的输出会在本地储存

分区 (Partitionnement): map 的输出会按键进行分区，将每个键映射到某个 Reduce 任务（每个键都会分给唯一一个 reduce）

reduce 端:

拷贝 (copy): 根据分区结果从 map 中下载属于自己的那部分数据

合并 (merge): 合并来自于多个 map 的数据，并将相同键的所有值整合成一个值列表

排序 (sort): 对合并后的数据进行排列，使其按照某个定义好的顺序被处理

la phase de reduce

通过 shuffle 以<键, 值>形式读取 map 的数据
处理
以<键, 值>形式输出数据

聚合中间结果

调用 Reduce 函数
输出结果
输出存储

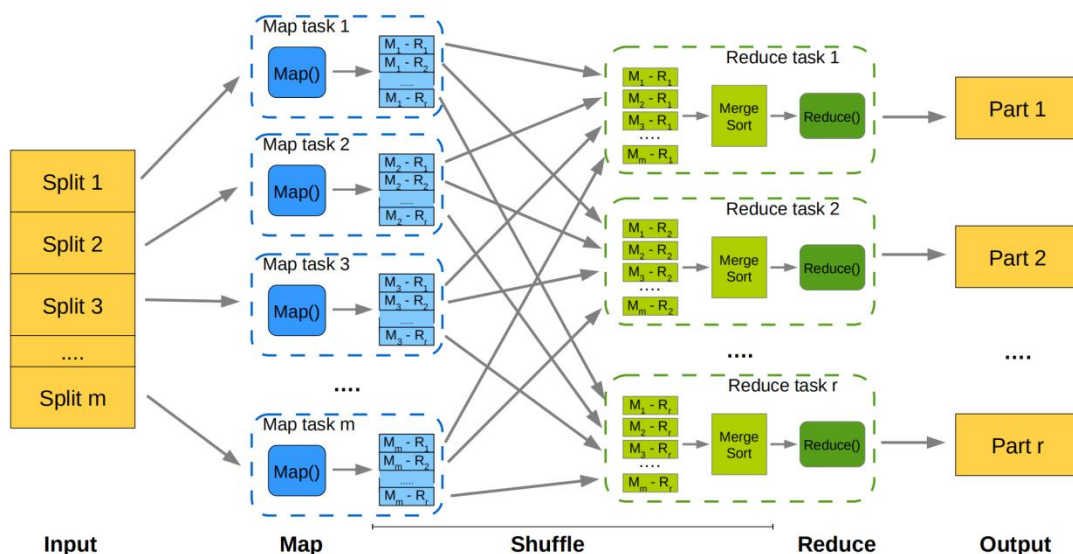
Reduce 任务的数量通常由用户静态定义，用户可以根据需求设置有多少个 Reduce 任务来处理中间结果。

$\text{Reduce} : (K2, \text{list}(V2)) \rightarrow \text{list}(K3, V3)$

(通常 $K2 == K3$)

至少提供 map 函数和 reduce 函数才能使其正常运行。

IMPORTANT



M_i : 第 i 个 Map 任务 生成的中间键值对。

R_j : 这些键值对被传递到第 j 个 Reduce 任务 进行处理。

map-reduce 执行一定需要:

一个 map 函数
一个 reduce 函数
数据

可配置区域:

Shuffle 阶段的分区策略: 如何根据键将中间结果分配给不同的 Reduce 任务的策略。

Shuffle 阶段的排序策略: Shuffle 过程中如何对键值对进行排序。

Reduce 任务的数量: 可以配置要运行多少个 Reduce 任务。

Split 切分策略: 如何划分输入数据成多个 Split, 供 Map 任务处理。

输入和输出的格式: 数据的输入格式和输出格式, 例如文本格式、二进制格式等。

Map-Reduce 结构可以多个连用，一个 MP 处理完数据后发往下一个 MP。

与数据库管理系统 SGDB 对比：

	SGBD classique	Map-Reduce
Taille	GigaOctets	PetaOctets
Accès	Interactif et batch	batch
Mises à jour	Plusieurs lectures et écritures	Une seule écriture, plusieurs lectures
Données structurées	oui	non
Passage à l'échelle	non linéaire	linéaire

Hadoop

开源系统 apache 开发

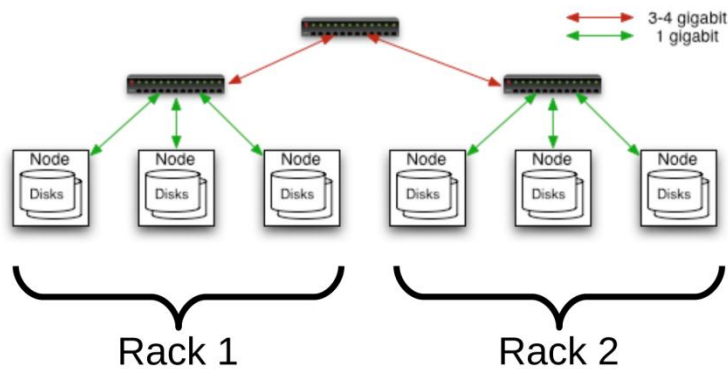
三大组件

HDFS 分布式文件系统

YARN 资源管理和任务调度系统

Map Reduce 编程模型和处理框架

架构：



集群中的机器按机架分组，通过网络交换机进行通信。机架内的通信相对快速，而机架之间的通信通常较慢。

所有机器组成 rack

在机栅 grille de machine 上运行

HDFS

Système de fichiers distribué, tolérant aux pannes et conçu pour stocker des fichiers massifs (> To).

分布式容错文件系统，专为存储海量（>To）文件而设计。

文件分布在 **128Mb**（默认大小）的块上

适配单个大文件

不适配多个小文件

一写多读

顺序读取：读取文件时，系统会更注重吞吐量（**debit**）而非延迟（**latence**）

单次数据读取的延迟较高

只支持附加写（**append-only**）并且互斥访问

文件系统只允许在文件末尾附加数据，而不能修改已经存在的数据。

文件不能任意修改，无法同时有多个写入者，并且不能在文件的任意位置修改内容。

物理块存储：文件在 **HDFS** 中被分割成多个块，实际存储在 **DataNode** 的本地文件系统上。

块副本（**réplication**）：每个块会被复制到多个 **DataNode** 上，默认情况下每个块有 **3** 个副本。

副本分布：为了保证数据的冗余性和可靠性，块的副本被分布存储在不同的 **DataNode** 上。

如果某个 **DataNode** 失效，文件仍然可以从其他 **DataNode** 的副本中恢复。

主 **jvm** 中 **NameNode** 核心管理节点 - 管理数据块的分配、分发和复制，和外界接口，公开树状命名空间

每个文件和托管数据节点的区块列表

每个区块的数据节点列表

文件参数

HDFS 的文件数量受 **NameNode** 内存大小限制

支持操作：读写增删

从 **jvm** 中 **DataNode** 数据储存节点 - 实际存储文件的内容（数据块），**HDFS** 客户端的数据块和元数据服务器，
维护自有数据块上的元数据

与 **NameNode** 交流

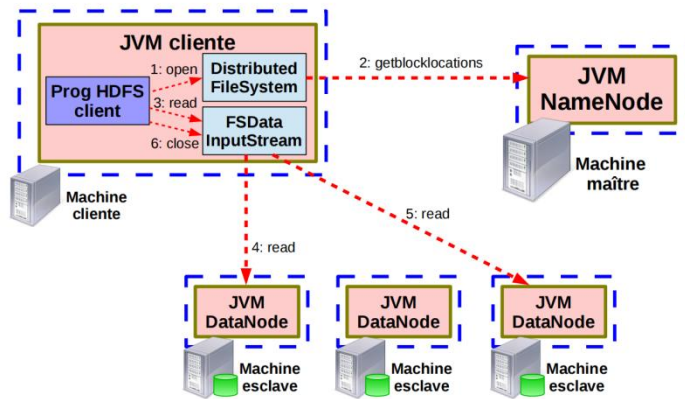
心跳 - 发出 - 总容量，所用所剩空间

接收 - 命令 - 向其他 **datanode** 复制块

- 废除块

定期通知 **Namenode** 本地包含的块

读取文件流程：



1.客户端发起请求（open）

客户端的 HDFS 程序（Prog HDFS client）通过 DistributedFileSystem 调用 open() 方法，请求读取某个文件。

2.向 NameNode 获取数据块位置（getBlockLocations）

客户端先联系 NameNode（主节点），询问：

“这个文件被切成了哪些数据块（block），每个块在哪些 DataNode 上有副本？”

NameNode 不返回数据本身，而是返回块的位置列表。

3.客户端开始读取（read）

客户端根据 NameNode 给的位置列表，决定先从哪个 DataNode 读第一个数据块。

（通常会优先选择距离最近的 DataNode，提高读取速度）

4.从第一个 DataNode 读取数据（read）

客户端直接通过 TCP 向 DataNode 请求数据块内容。

数据不是经过 NameNode 中转的，而是直接从 DataNode 发给客户端。

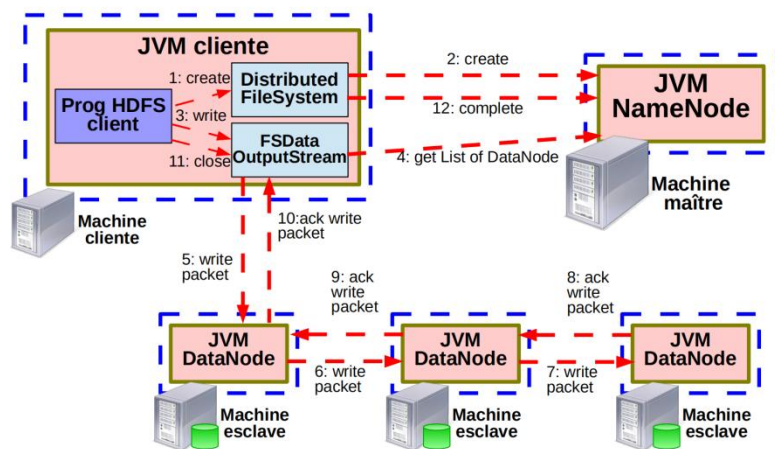
5.如果第一个 DataNode 不可用

客户端会自动切换到列表中的下一个 DataNode（读副本），继续读取，保证容错性。

6.读完后关闭连接（close）

当文件所有数据块都读取完成，客户端调用 close() 关闭流，读取流程结束。

写入文件流程：



1. 客户端发起创建请求（create）

客户端 HDFS 程序（Prog HDFS Client）通过 DistributedFileSystem 调用 create() 方法，告诉 HDFS 要写一个新文件。

2. 通知 NameNode 创建文件

客户端把“我要创建这个文件”的请求发给 NameNode（主节点）。

NameNode 检查：

目录是否存在

文件是否已存在

客户端是否有权限写

如果一切正常，NameNode 先在元数据里登记这个文件的创建操作（但此时文件还是空的）。

3. 客户端开始写数据（write）

客户端将数据通过 FSDDataOutputStream 分成一个个数据包（packet）。

每个数据块（block）通常是 64MB 或 128MB，一个数据块又被拆成多个 packet 发送。

4. 获取 DataNode 列表（get List of DataNode）

NameNode 返回一组 DataNode 的列表（通常是 3 个，代表这个块的副本存放位置）。

比如：DataNode1 → DataNode2 → DataNode3（这就是副本链的顺序）。

5. 写入第一个 DataNode（write packet）

客户端把 packet 先发给列表中的第一个 DataNode。

6. 第一个 DataNode 转发给第二个 DataNode

DataNode1 收到数据后，一边写入本地磁盘，一边转发给 DataNode2。

7. 第二个 DataNode 转发给第三个 DataNode

DataNode2 同样写入磁盘，并转发给 DataNode3。

8. 第三个 DataNode 确认写入完成 (ack)

DataNode3 完成写入后, 返回一个确认 (ack) 给 DataNode2。

9. 第二个 DataNode 确认写入完成 (ack)

DataNode2 收到 DataNode3 的 ack 后, 再向 DataNode1 发 ack。

10. 第一个 DataNode 确认写入完成 (ack)

DataNode1 收到 DataNode2 的 ack 后, 向客户端发 ack。

11. 客户端关闭文件 (close)

客户端调用 close(), 告诉系统数据已经全部写完。

12. 通知 NameNode 写入完成 (complete)

客户端发 complete 给 NameNode, NameNode 更新元数据, 标记这个文件写入成功。

HDFS 块放置策略

一个副本存放在本地机架 (rack local)

第二个副本存放在另一个机架上 (un autre rack)

第三个副本存放在另一个数据中心的机架上 (un rack d'un autre datacenter)

额外的副本会随机放置 (réplicas supplémentaires sont placés de façon aléatoire)

HDFS 容错

NameNode 崩溃:

服务不可用

DataNode 崩溃:

数据仍然可用

NameNode 不在收到对应的心跳

NameNode 崩溃的解决方案

在稳定介质上预防性备份事务日志，重启后的 NameNode 可以加载最新的 HDFS 映像并应用事务日志。

建立一个能自动取代原有名称节点的备份名称节点：

使用 ZooKeeper 检测 NameNode 故障并通知备份 NameNode

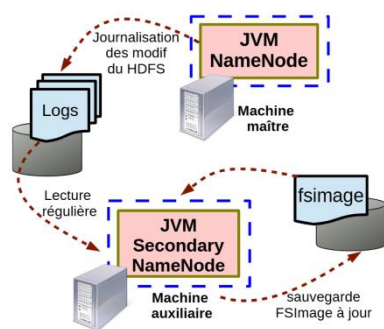
NameNode 的重启很少发生

日志存储量巨大

重启时间长

解决方法：

使用一个或多个 SecondaryNameNode 来定期生成元数据快照



SecondaryNameNode 不是 NameNode 的热备份，只能定期合并快照和日志

NameNode 挂了，需要手动使用 SecondaryNameNode 生成的 fsimage，启动一个新的 NameNode

HDFS 联邦 (Fédération de HDFS)

目的:

为多个 HDFS 实例互用同一个数据节点:

⇒ 多个命名空间之间的隔离

⇒ 名称节点之间的负载平衡

hdfs dfs <命令>

YARN (Yet Another Resource Negotiator)

Service de gestion de ressources de calcul distribuées

分布式计算资源管理服务

主节点 - ResourceManager (RM)

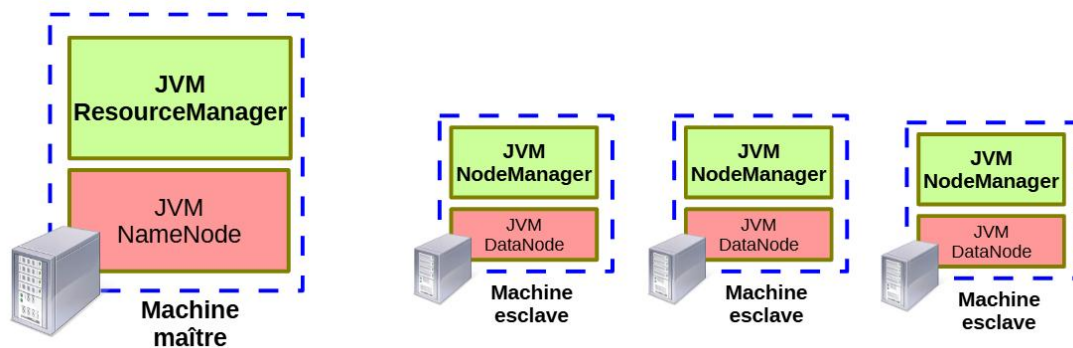
控制集群中的所有资源

调度客户端请求

从节点 - NodeManager (NM)

管理资源节点

启动和监控容器



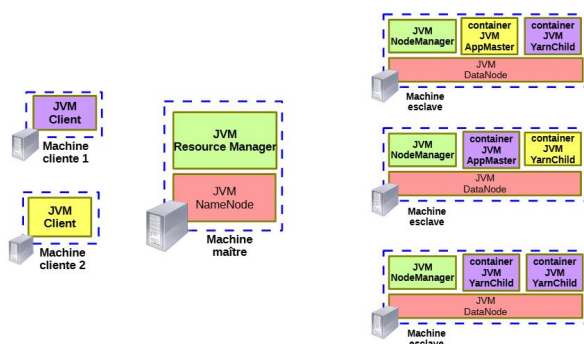
主容器 - ApplicationMaster (AM)

应用程序的主容器

与 ResourceManager 协商

从容器 - YarnChild (YC)

执行应用任务



ResourceManager 的目标:

- 协调集群中的守护进程
- 决定应用程序容器的分配位置
- 外部和内部组件的联系
- 系统全局状态的视图
- 提交应用程序

RM 对外的工作 (Interface avec l'extérieur) :

(1) 处理客户端请求

提交应用 (soumission d'applications) :

分配、部署、启动 ApplicationMaster 容器

→ RM 给你找个机器, 分配资源, 并启动 AM。

AM 再去申请 YarnChild 任务容器

→ 具体的任务 (Map/Reduce 等) 由 AM 向 RM 要资源, 然后启动 YarnChild 去执行。

删除应用 (suppression d'applications) :

释放该应用占用的所有 container 资源 (让别的任务可以用)。

提供当前提交状态信息:

比如你可以查到一个应用正在运行、已完成、失败等状态。

NodeManager 的角色:

在本机托管并启动任务容器 (container)

按 RM 或 AM 的要求释放容器资源 (任务结束、被取消等)

监控本机物理节点和容器的健康状况 (CPU、内存、磁盘、进程存活)

保存 RM 下发的安全密钥, 确保只有合法的容器才能运行 (防止越权访问)

ApplicationMaster 的角色:

负责与 ResourceManager 协商资源

协作管理容器

管理应用生命周期

YarnChild 容器的角色:

执行应用程序任务

NodeManager 与 ResourceManager 的通信模式：

心跳机制（Heartbeat）：

发送的信息

已托管的容器状态

节点状态

返回的信息：

分配的会话密钥

容器释放指令

其他指令：如停止任务或重新同步。

ApplicationMaster 与 ResourceManager 的通信模式：

心跳机制（Heartbeat）：

发送的信息：

应用程序进展

容器请求

容器释放

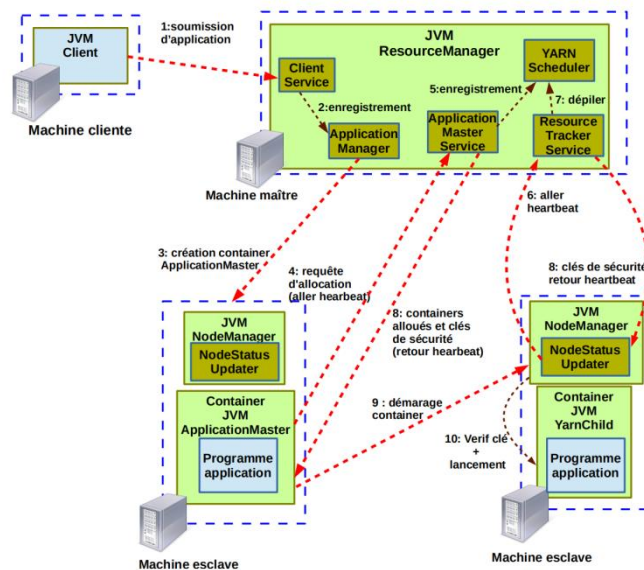
返回的信息：

新分配的容器列表

可用资源

停止或同步的指令

部署示意图



0. 谁是谁

- **RM** = ResourceManager (资源大总管)，图中大绿框里的几块：
 - **ClientService**: 收客户端请求
 - **ApplicationMasterService**: 收 AM 的注册与资源申请
 - **ResourceTrackerService**: 收 NodeManager 心跳
 - **Scheduler** (Capacity/Fair 等)：真正做资源分配
- **NM** = NodeManager (每台机的小管家)，里面有 **NodeStatusUpdater** (发心跳)
- **AM** = ApplicationMaster (应用的项目经理)，运行在一个 **Container** 里
- **YarnChild** (MRv2 里的具体 Task 进程；像 Spark 则是 Executor)

1) soumission d'application (应用提交)

客户端 (Client) 把应用提交到 RM 的 **ClientService**: 携带 AM 需要的资源、启动命令、凭证等。

2) enregistrement (登记/受理)

RM 的 **ApplicationsManager/ClientService** 接单: 生成 ApplicationId, 入队等待调度。

3) création container ApplicationMaster (为 AM 分配并启动容器)

Scheduler 选一台合适的节点, RM 通过该节点的 **NodeManager** 启动一个 **Container**, 在里面起 **ApplicationMaster** 进程。

注：这一步常通过 NM 的心跳下发“启动 AM 容器”的指令。

4) requête d'allocation (aller heartbeat) (AM 发资源请求/心跳)

AM 启动后，周期性调用 RM 的 **ApplicationMasterService**:

- 先 `registerApplicationMaster()` (有的图把“注册”画在 5)；
- 再在每次 `allocate()` 心跳里申请运行任务所需的若干 **Container** (比如 Map/Reduce 或 Spark Executor)。

5) enregistrement (AM 向 RM 注册)

AM 与 RM 完成注册, RM 知道“这个应用的项目经理上线了, 可以给它分资源了”。

6) aller heartbeat (NM → RM 心跳)

各台 **NodeManager** 周期性向 RM 的 **ResourceTrackerService** 报平安: 资源余量、容器状态、故障等。

7) dépiler (调度出队)

Scheduler 根据队列策略 (Capacity/Fair/FIFO)、资源情况与 AM 的请求, 做出分配决策: 哪些节点给哪些容器。

8) containers alloués et clés de sécurité (retour heartbeat)

RM 在回复 NM 的心跳里下发指令:

- “给某应用分了这些 **Container**”;
- 同时带上安全凭证 (**ContainerToken/NMToken**, 以及需要时的 HDFS 访问票据)。

9) démarrage container (NM 启动容器)

指定的 **NodeManager** 拿到分配与令牌后, 拉起对应 **Container**, 准备运行实际任务进程。

10) Vérif clé + lancement (校验证书并启动任务)

容器侧验证令牌, 加载用户程序与依赖, 然后启动:

- 在 MapReduce 上就是 **YarnChild** (具体的 Map/Reduce task attempt)；
- 在 Spark 上就是 **Executor** 进程。

后续 AM 会继续在 4/7/8 这个三步回路里要资源、拿分配、起容器, 直到任务全部完成。

YARN: 不同的调度策略

FIFO Scheduler: FIFO（先进先出）调度器按照请求的到达顺序处理任务。

优点：非常简单。

缺点：集群资源的分配效率低，因为没有考虑优先级和资源利用率。

Capacity Scheduler: 容量调度器使用多个带有最大容量限制的分层队列来分配资源。

优点：小请求不会被大的任务延迟。

缺点：并非所有资源都能被充分利用。

Fair Scheduler: 公平调度器为每个任务分配相同的资源，并根据需要进行动态调整。

优点：动态分配资源，不需要等待。

缺点：随着任务数增加，分配给每个任务的资源会减少。

YARN: ResourceManager 的容错性

预防措施:

定期保存 **ResourceManager** 状态: 所有提交应用程序的列表及其元数据，集群中从节点的状态。

ResourceManager 重启时的处理过程:

第一阶段: 恢复应用程序列表和从节点状态，这些信息来自之前保存的稳定存储介质。

第二阶段: 逐步恢复所有应用程序的执行上下文，

通过: 读取保存的元数据。

从节点的心跳恢复应用和容器状态。

YARN: 从节点崩溃的容错性

对 **NodeManager** 的影响:

如果 **NodeManager** 不再向 **ResourceManager** 发送心跳信号:

ResourceManager 将删除该节点，直到它重新启动。

对 **ApplicationMaster** 的影响:

如果 **ApplicationMaster** 容器停止向 **ResourceManager** 发送心跳信号:

ResourceManager 会启动新的 **ApplicationMaster** 实例以恢复应用程序。

对 **YarnChild** 的影响:

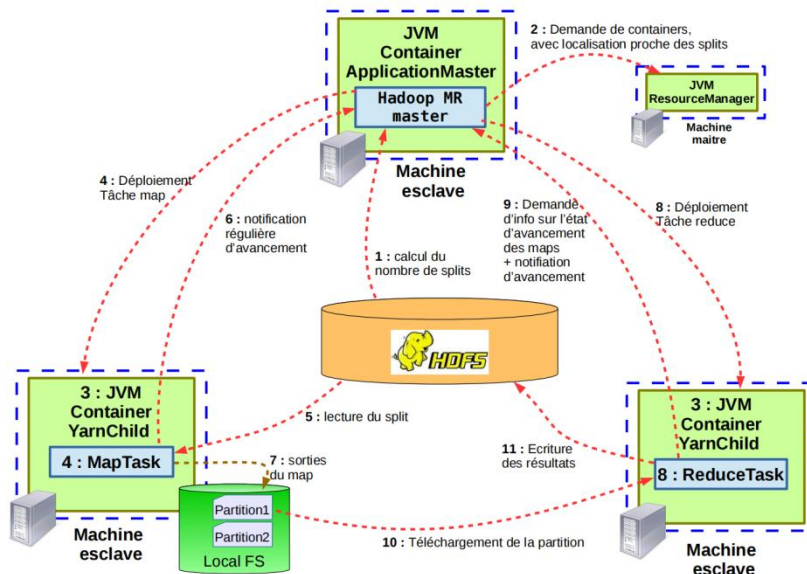
如果 **YarnChild** 容器无法与 **ApplicationMaster** 通信:

Hadoop MapReduce

Appli Yarn pour l'exécution et le déploiement de programmes Map-Reduce

用于运行和部署 Map-Reduce 程序的 Yarn 应用程序

流程:



1. 计算输入分片（splits）数量

ApplicationMaster（Hadoop MR master）启动后，先去 HDFS 查看输入数据，计算要切成多少个 split（分片）。

每个 split 会交给一个 MapTask 处理。

2. 向 ResourceManager 申请容器（Container）

AM 申请运行 MapTask 所需的容器，并尽量要求这些容器靠近数据存储位置（数据本地化，减少网络传输）。

3. ResourceManager 分配容器

RM 分配合适的节点，返回给 AM。

4. 部署 MapTask

AM 在分配到的容器里（JVM YarnChild）启动 MapTask 进程。

5. MapTask 读取 split

MapTask 从 HDFS 读取自己负责的 split 数据。

6. 定期发送进度

MapTask 定期向 AM 报告执行进度（心跳 + 状态），方便 AM 监控任务情况。

7. Map 输出到本地

MapTask 处理完后,会把输出结果分区(Partition1, Partition2...)写到本地文件系统(Local FS),为后续 Reduce 做准备。

8. 部署 ReduceTask

当部分或全部 MapTask 完成后,AM 向 RM 再申请容器,部署 ReduceTask。

9. Reduce 进度监控

AM 会向 ReduceTask 查询 Map 的进度,并同步整体任务状态。

10. Reduce 拉取数据 (Shuffle)

ReduceTask 从各个 MapTask 的本地磁盘拉取自己需要的分区数据(跨节点传输)。

11. Reduce 写结果

ReduceTask 处理完成后,将最终结果写回 HDFS。

YarnChild 容器的职责：

执行任务：YarnChild 容器负责执行单个 map 或 reduce 任务实例。

报告进度：容器将任务的进度报告给对应的 ApplicationMaster。

AM 容器的职责：

Map-Reduce 作业的协调：

在创建时定义需要请求的容器数量。

容器请求会根据作业输入的数据位置来考虑数据的本地性。

在 RM (ResourceManager) 分配的容器中部署任务。

Map 任务：尽量部署在接近数据分片的位置。

Reduce 任务：当一定比例的 map 任务完成后启动。

推测执行 Spéculation

AppMaster 决定是否启动推测任务：当判断任务进展过慢时，AppMaster 可以决定启动新的任务实例来加速执行。

推测任务的启动：如果推测执行是必要的，新的任务实例会被分配到一个容器中执行。

同一个任务的多个实例可能同时执行。如何保证 HDFS 文件的一致性？

每个实例写入到一个临时文件。

AppMaster 指定一个实例（通常是最早完成的实例）将结果写入到 HDFS 的最终位置。

其他实例及其数据在完成写入后被销毁。

scala

scalable language

val 不可变



var 可变

class 模板

object 单例

trait 特质：类似接口，但是可以实现方法

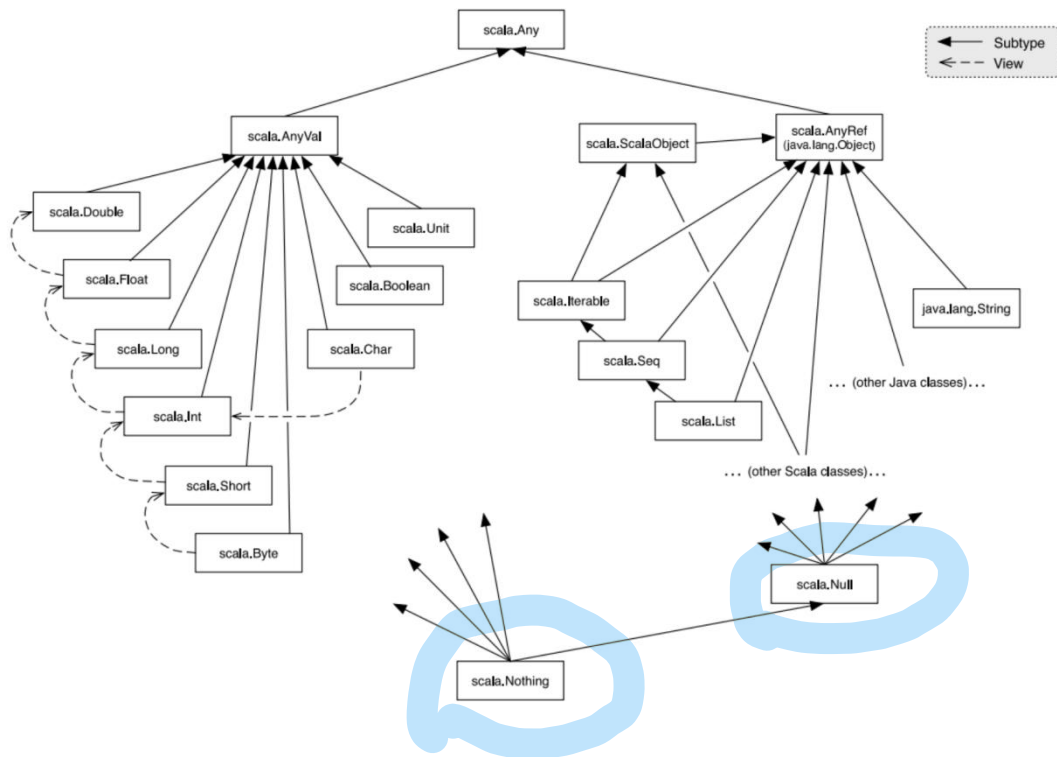
case class 样例类：储存不可变数据

nothing 是所有类型的子类型（既不是值类型，也不是引用类型）

Nil -> List[Nothing] 空列表

单个的 nothing 不能持有值（不能赋值），但是可以违异常抛出

null 是引用类型



所有操作符都是函数，而且还都可以重载

```
if (<exprBooleenne1>) EXPRESSION1  
  
  else if (<exprBooleenne2>) EXPRESSION2  
  
  else EXPRESSION3
```

没有 switch

```
while (<exprBooleenne>) {  
  ...//code a faire tant que exprBooleenne est vraie  
}
```

```
do{  
  ...//code a faire une fois a repeter  
  //tant que exprBooleenne est vraie  
}while(<exprBooleenne>)
```

```
for (x <- COLLECTION) EXPRESSION
```

Pour une sequence de valeurs entières

```
for (i <- Range.apply(MIN,MAX,STEP) ) { /*code pour valeur i*/}
```

/*ou bien */

```
for (i <- MIN to MAX by STEP ) { /*code pour valeur i*/ }
```

Pour une table associative

```
for ((k,v) <- my_map ) { /*code pour la cle k et la val v*/ }
```

没有 break 没有 continue

需要使用 break 类

```
val outer = new Breaks;  
val inner = new Breaks;  
  
outer.breakable {  
  for( a <- 1 to 10){  
    inner.breakable {  
      for( b <- 1 to 20){  
        if( b == 12 ){  
          inner.break;  
        }  
      }  
    }  
  }  
} // inner breakable  
}  
} // outer breakable
```

函数可以定义在函数内部

抽象函数 没有返回体（没=），子类必须实现（使用 `override def xxxx`）

构造函数直接写在类中

辅助构造函数需要调用另一个辅助构造函数或者主构造函数

辅助构造函数≈java 的重载构造函数

```
class Point(x: Int, y: Int){  
    def this() = this(0,0)  
    def this(x: Int) = this(x,0)  
    ...  
}
```

`_Seq()` set 方法（只有 `var` 有）

`final class`: 禁止子类化 - 不能被继承

`sealed class`: 受限继承 - 子类必须在与父类同一个文件中

```
tous les membres de p : import p._  
le membre x de p : import p.x  
le membre x de p renommé a : import p.{x => a}  
les membres x et y de p : import p.{x, y}  
un membre interne z d'un de membre p2 de p : import p.p2.z
```

只有 `private` 可以修饰类，`protected` 不行

有抽象类 和 java 的一样

```
class C extends P.B{
  def c(ca:P.A, cb:P.B, cc:C)=...
}
```

[illegible]

trait 和 interface 区别

1 继承能力

特性	Scala <code>trait</code>	Java <code>interface</code>
能否继承类	✅ 可以继承单个 类 (但要注意 <code>trait</code> 只能继承一个具体类, 其他必须是 <code>trait</code>)	❌ 只能继承 <code>interface</code> , 不能继承 <code>class</code>
能否多继承	✅ 可以同时继承多个 <code>trait</code>	✅ Java 8+ 接口也可以多继承 (但仅限接口)

2 成员类型

特性	Scala <code>trait</code>	Java <code>interface</code>
普通字段 (变量)	✅ 可以有 <code>val</code> (不可变) 和 <code>var</code> (可变) 字段	❌ 不能有实例字段, 只能有 <code>public static final</code> 常量
方法实现	✅ 从一开始就可以有方法实现 (类似抽象类)	Java 8 之前 ❌; Java 8 之后 ✅ (<code>default</code> 方法)

3 构造器

特性	Scala <code>trait</code>	Java <code>interface</code>
构造代码块	✅ 可以有构造代码块 (在混入时执行)	❌ 不允许构造代码块
构造参数	❌ 不能直接在 <code>trait</code> 定义构造参数 (需要抽象成员变量)	❌ 同样不能有构造参数

`sealed + trait` 表示只能在当前文件中被继承/扩展

```
class Point(var x: Int, var y: Int)
  extends Movable (trait/class)
  with Drawable (trait)
  with xxxx (trait)
```

case 类： 自带 equals, hashCode, copy, toString 等
自带伴生对象（apply/unapply 方法）
实例化的时候不用 new（因为 apply）

T <: Fruit 泛型类型 T 必须是 Fruit 类型或者是 Fruit 的子类

T >: Machin 泛型类型 T 必须是 Machin 类型或者是 Machin 的父类

T <% Legume 类型 T 必须可以被隐式转换为 Legume 类型

协变 +T: 允许子类泛型实例赋值给父类泛型实例（用于只读操作）。

逆变 -T: 允许父类泛型实例赋值给子类泛型实例（用于只写操作）。

协变，如果 A 是 B 的父类，那 C (A) 是 C (B) 父类。

逆变，如果 A 是 B 的父类，那 C (B) 是 C (A) 父类。

函数可以像变量一样赋给其他变量

可以作为参数传递

函数的返回值也可以是函数

函数返回值类型可以省略

args : Int* 传入多个 int 作为参数

def name (a : => Long , b : => Long) 按需参数，只有使用时才会计算

隐式转换：implicit

隐式参数：implicit （作为参数，调用函数的时候编译器会自动找个合适的值填进去）

3. 典型用法

(1) 给不存在的方法开外挂

```
scala📄 复制 ✎ 编辑  
  
class RichInt(val value: Int) {  
  def square: Int = value * value  
}  
  
implicit def int2RichInt(x: Int): RichInt = new RichInt(x)  
  
println(5.square) // 编译器看到 Int 没有 square 方法 → 调用 int2RichInt(5).square
```

(2) 自动类型适配

```
scala📄 复制 ✎ 编辑  
  
implicit def doubleToInt(d: Double): Int = d.toInt  
  
val x: Int = 3.14 // 自动变成 3
```

option[A] 一个可能有值，也可能没值的容器

```
def racine(d: Double): Option[Double] =
  if (d >= 0) Some(math.sqrt(d))
  else None
```

- 如果 `d` 是非负数，返回平方根的 `Some` 包装；
- 否则返回 `None`。

集合：

Les Set
Collections sans doublons.

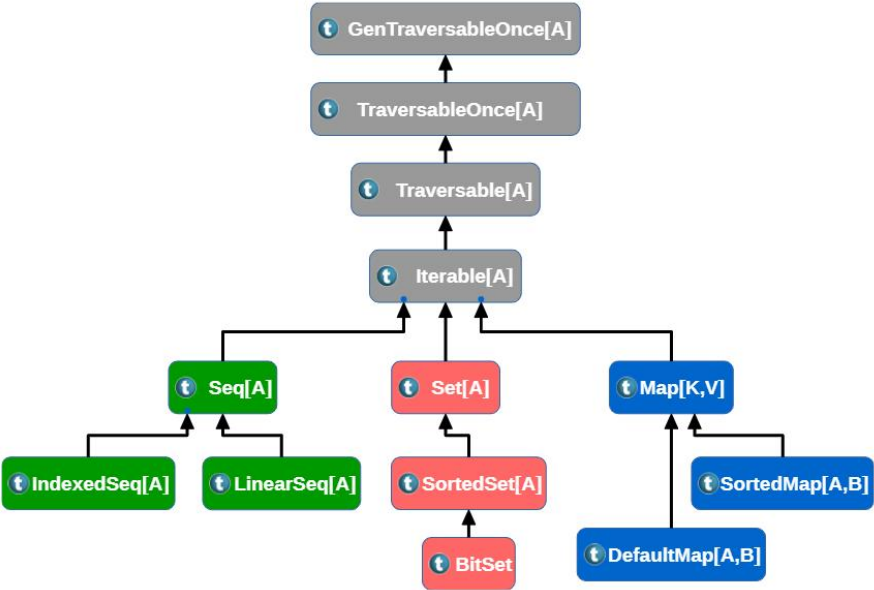
Les Seq
Collections associant à chaque élément un indice de position de 0 à `size - 1`
⇒ Doublons possibles

Les Map
Collections de couples (clé, valeur) assurant l'unicité des clés. Un élément peut s'exprimer avec la syntaxe :

- `(key, value)`
- ou bien `key -> value`

	val 不可变	var 可变
可变集合（mutable）	集合 -= 元素	集合 -= 元素
不可变集合（immutable）	无	集合 = 集合 - 元素

增加元素同上



函数式编程：

Concevoir des programmes comme des fonctions mathematiques que l'on compose entre elles.

将程序设计成数学函数，并将它们组合在一起。

Exploiter des fonctions avec des proprietes particulieres augmentant la composition/decomposition de fonctions

利用具有特殊属性的函数来增强函数的组合/分解。

可变参数（arg: xxx *），重点在*

部分应用函数（预先固定一个/多个参数，形成一个新的函数，新的函数只需要传入剩余的参数即可）：

```
def log(date: Date, mess: String) = println(date + ": " + mess)
val now = new Date
val log_with_predefined_date = log(now, _: String)
```

```
log_with_predefined_date("mess1")
```

```
log_with_predefined_date("mess2")
```

高阶函数：能把函数当作参数传进去，或者能返回一个函数的函数

柯里化函数（将单个<多个参数>的函数转化为多个<单个参数>的函数）：

```
def uncurried_add(x: Int, y: Int) = x + y
def curried_add0(x: Int)(y: Int) = x + y
```

```
val addWith3 = curried_add0(3) // 部分应用，得到一个新的函数 (y: Int) => 3 + y
println(addWith3(5)) // 输出：8
```

匿名函数版柯里化：

```
def curried_add1(x: Int) = (y: Int) => x + y
```

嵌套版匿名函数柯里化：

```
val curried_add2 = (x: Int) => (y: Int) => x + y
```

隐式参数：

```
def foo(hours: Int)(implicit amount: BigDecimal, curName: String) =
    (amount * hours).toString() + " " + curName
```

```
implicit val hourlyRate = BigDecimal(34)
```

```
implicit val currencyName = "Dollars"
```

```
val x = foo(3)
```

Spark

RDD - 弹性分布式数据集 只读

val id: Int （spark 分配）

var name: String （可以人为命名）

包含分区信息

partitions: Array[Partition]

→ 当前 RDD 有哪些分区，每个分区的对象信息。

getNumPartitions: Int

→ 分区数量。

preferredLocations(split: Partition): Seq[String]

→ 每个分区在哪些节点上优先存储（比如 HDFS block 的位置），Spark 会尽量在本地计算，减少网络传输。

spark 代码执行四个步骤

1. 初始化 sparkContext
2. 创建 RDD
3. RDD 之间转换
4. 在最终 RDD 上执行 Action

窄依赖（一对一）：

.filter(_.length < 6)

```
rdd.map(_.split(" ")).flatMap(x => x)
= rdd.flatMap(line => line.split(" "))
```

.union() 并集（合在一起）（也可以用 rdd1++rdd2）

.zip() 合并（把俩 rdd 中的分区内容一对一合在一起）（分区数不一样就会报错）
（对应分区中元素个数也必须相同）

keyBy 操作

作用：根据一个函数，将 RDD 中的每个元素转换为键值对。键由函数生成，值为元素本身。

zipWithIndex()

给 rdd 中的元素加个序号，从 0 开始

宽依赖（一对多）：

Intersection（rdd，分区数）

取交集

distinct(分区数)

去重

subtract 操作

作用：返回一个 RDD，包含调用 RDD 中存在而在另一个 RDD 中不存在的元素。

cartesian 操作

作用：cartesian 返回两个 RDD 的笛卡尔积，即每个元素与另一个 RDD 中每个元素配对。

sortBy(函数，升降序 true/false)操作

作用：sortBy 根据指定的键函数对 RDD 的元素进行排序。

sortBy(s => s, true) 按字母顺序排列

repartition 操作

作用：repartition 改变 RDD 的分区数，返回一个新的 RDD。分区数可以增加或减少。

cogroup

将两个 rdd 按键配对=》<键，<值 1>，<值 2>>

以 key 为单位，把两个 RDD 中相同 key 的所有 value 分别收集成两个列表(Iterable)

groupByKey

按键合并值

reduceByKey(分区，函数)

join

按 key 把两个 RDD 中 key 相同的元素配对

(K, (V, W)) 当左 rdd 和右 rdd 的值不同时，会发生笛卡尔积

常见的简单 Action 操作：

`max()` 和 `min()`：返回 RDD 中的最大和最小值。

`isEmpty()`：判断 RDD 是否为空，返回布尔值。

`first()`：返回 RDD 中的第一个元素。

`count()`：返回 RDD 的总元素个数。

`collect()`：收集 RDD 的所有元素并返回为数组。注意：通常仅用于调试或在小数据集上使用，避免在大型数据集上使用。

`take(num: Int)`：返回 RDD 的前 num 个元素。

`def foreach(f: (T) => Unit): Unit` 对 RDD 的每个元素 应用一个函数 f

`def reduce(f: (T, T) => T): T` 用一个 结合律（**associative**）且交换律（**commutative**）的二元函数 f 把 RDD 的所有元素合并成一个值。常用来做加总、最大值、最小值等聚合。

`saveAsObjectFile(url_file: String)`：将 RDD 保存为对象文件，以序列化格式存储在指定 URL 路径。

`saveAsTextFile(url_file: String)`：将 RDD 的内容以文本格式保存为文件。

持久化储存级别：

MEMORY_ONLY_SER 把 RDD 序列化（**serialized**） 存在内存中。

优点：

存储量更省（因为序列化后数据更紧凑）。

读取速度快（内存访问）。

缺点：

如果内存放不下，多余的分区会 丢失，需要重新计算。

适用场景：内存有限，但希望节省空间，并且重新计算代价不大。

MEMORY_AND_DISK / MEMORY_AND_DISK_SER

先把 RDD 存到内存，如果内存不够，就把溢出的数据放到本地磁盘。

优点：

不会丢数据，哪怕数据很大。

缺点：

读写磁盘比内存慢。

SER 版本是序列化后再存储，节省空间，但需要反序列化开销。

DISK_ONLY

完全存到磁盘。

优点：

能保存超大数据集。

缺点：

延迟高（I/O 慢）。

适合：数据非常大、内存有限、但需要避免重复计算。

额外选项 _2

例如 MEMORY_ONLY_2、MEMORY_AND_DISK_2

意思：每个分区会在 两个节点 上保存一份（数据副本）。

好处：容错性高，一个节点挂了还能从另一个节点读。

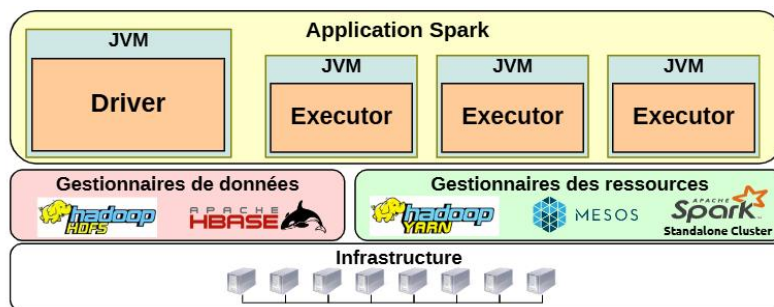
坏处：占用存储空间更多。

丢失数据之后，

无持久化->从最初的 rdd 开始重算

有持久化->会从生成这个 rdd 的 rdd 中拿数据，然后再处理，以恢复数据

架构



Driver 主控进程-分配任务

Executor 工作进程-执行任务

执行流程

1. 初始化 SparkContext (SparkContext)

Driver 里 `new SparkContext(...)`，加载配置，准备元数据。

2. 创建首批 RDD (Creation of first RDDs)

比如 `sc.textFile(...)`。Driver 去数据管理器(HDFS/HBase)问数据分片位置 (split/data locality)，为每个 split 规划一条根任务 (1 split $\approx$ 1 task)。

3. RDD 变换 $\rightarrow$ 构建 DAG (Transformations $\rightarrow$ DAG)

你写的 `map / reduceByKey / filter ...` 被翻译成有向无环图 (DAG)。调度器会按宽/窄依赖把 DAG 切成 Stages，尽量优化数据传输。

4. 触发 Action (Job 执行·上半场)

碰到 `saveAs.../count/reduce` 等 Action 才真正开跑：

Driver 向资源管理器 (ResourceManager: YARN/Mesos/Standalone) 请求启动 Executors。

5. 分配与执行 (Job 执行·下半场)

- Driver 把各个 Stage 的 Tasks 派给 Executors，跟踪“完成/失败”。
- 失败会重试；需要的话把中间结果按你的 `persist/cache` 策略缓存在 Executor 节点。

6. 结果去向 (Job 执行·收尾)

- 如果是 保存型 Action (`saveAsTextFile` 等)：Executor 把结果写回存储系统。
- 如果是 返回结果的 Action (`reduce` 等)：结果回传给 Driver。

广播变量：

```
val broadcastVar = sc.broadcast(Array(1, 2, 3))  
  
broadcastVar.value // permet d'accéder à la variable
```

累加器

```
val accum = sc.longAccumulator("MyAccu")  
sc.parallelize(Array(1, 2, 3, 4)).foreach(accum.add(_))  
accum.value // permet d'accéder à la valeur de l'accumulateur
```

Streaming

单个值

来自 Spark Core 的通用转换

- **mapPartitions**：按分区对数据做映射处理（比 map 粒度更粗）。
- **filter**：过滤满足条件的元素。
- **flatMap**：映射后“扁平化”展开（一个输入 → 多个输出）。
- **glom**：将一个分区的元素合并成一个数组。
- **repartition**：重新分区。
- **union**：合并两个 DStream。
- **map**：对每个元素做映射处理（一个输入 → 一个输出）。

Spark Streaming 特有的转换

- **count**：计算每个批次中的元素个数。
- **countByValue**：统计每个值出现的次数。
- **countByValueAndWindow**：按滑动窗口统计值的出现次数。
- **reduce**：聚合操作（批次内）。
- **transform**：用任意 RDD → RDD 函数转换 DStream（可用 Spark Core API）。
- **countByWindow**：按滑动窗口统计元素总数。
- **reduceByWindow**：按滑动窗口进行 reduce。
- **window**：基于滑动窗口重新生成 DStream。

键值对

来自 Spark Core 的键值对转换

- **fullOuterJoin / rightOuterJoin / leftOuterJoin / join**: 各种连接操作。
- **cogroup**: 将两个 RDD 按 key 进行分组。
- **combineByKey**: 自定义按 key 聚合。
- **flatMapValues**: 对 value 做 flatMap (不改 key) 。
- **groupByKey**: 按 key 分组。
- **mapValues**: 只对 value 做 map (不改 key) 。
- **reduceByKey**: 按 key 聚合 (减少) 。

Spark Streaming 特有的键值对转换

- **groupByKeyAndWindow**: 按 key 和时间窗口分组。
- **reduceByKeyAndWindow**: 按 key 和窗口进行聚合。
- **updateStateByKey**: 跨批次更新每个 key 的状态 (常用于计数器、会话跟踪等) 。

输出操作 (没有输出操作的话 dstream 的计算不会执行)

(1) 将每个批次的 RDD 写入文件

- **saveAsObjectFiles(prefix, suffix)**
将 RDD 保存为对象文件 (序列化后的二进制形式) 。
- **saveAsTextFiles(prefix, suffix)**
将 RDD 保存为文本文件 (每个元素一行) 。
- **saveAsHadoopFiles(prefix, suffix)**
按 Hadoop 文件格式保存。

(2) 在 Driver 控制台输出

- **print()**
打印每个 RDD 的前 10 个元素。
- **print(nb)**
打印每个 RDD 的前 `nb` 个元素 (自定义数量) 。

⚠ 适合调试用, 不适合大数据量输出。

(3) 通用操作 (Generic Operation)

- **foreachRDD(foreachFunc: RDD[T] => Unit)**
允许对每个 RDD 执行自定义逻辑, 比如:
 - 写入数据库 (MySQL、Cassandra、MongoDB...)
 - 发送到外部服务 (Kafka、Elasticsearch...)
 - 做额外的批处理分析

状态转换

- **updateStateByKey**

跨批次维护每个 key 的状态（例如累计计数、会话状态、余额变化等）。

```
scala
```

复制 编辑

```
val runningCounts = pairs.updateStateByKey(updateFunc)
```

- **window**

基于时间窗口组合多个批次的 RDD。

- **countByWindow**

在滑动窗口内统计元素总数。

- **countByValueAndWindow**

在窗口内统计每个值出现的次数。

- **reduceByWindow**

在窗口内对所有元素做 reduce 聚合。

- **reduceByKeyAndWindow**

在窗口内对每个 key 做 reduce 聚合。

- **groupByKeyAndWindow**

在窗口内按 key 分组。

```
def updateStateByKey[S](update: (Seq[V], Option[S]) => Option[S]): DStream[(K, S)]
```

用于跨批次维护每个 key 的状态

```
def window(windowDur: Duration, slideDur: Duration): DStream[T]
```

用于滑动窗口计算

避免 IO

便于定位直接计算的数据

便于传输数据

API 丰富

qdc: p24 intersection 之后的结果

其他函数之后的结果

les action 考试

p61

ZooKeeper

une outil de coordination de système distributés

分布式系统->难以同步

主从结构

主节点损坏->备用节点顶上

从节点损坏->主节点删除损坏从节点，并重新分配任务

网络损坏->从节点重新选举节点

每个要设计的系统都需要：

To Do (pour chaque système à concevoir)

- Mécanisme d'élection de leader
- Détecter les pannes/les nouvelles machines
- Stockage des méta-données du système
- Mécanisme de synchronisation fiable

arborescent 树状

presistant 持久

répliqué 多副本

séquentiellement cohérent 顺序一致性（全局顺序保证）

observable 可观察（订阅）

无法储存大量/有关系的数据

无法提供高一致等级的 api

特性：

client 必须建立连接并保持会话

系统所有操作都与客户端会话相关联

连接在任意选择的一个服务器上建立

一个会话涉及两个线程：

程序的主线程

一个用于 zookeeper 提醒/通知的线程

定期发送心跳以维持会话

键值数据：

znode 绑定一个可修改的值，无类型字节数组，大小上限为 1Mb

每个 **znode** 都有一个父节点，可能有一组子节点

znode 用绝对路径唯一标识，从根节点/开始

写入

set 修改

create 从父节点创建

delete 删除（前提是没有子节点）

读取

get 读取 **znode** 的值

ls/getChildren 列出子节点名称

exists 测试 **znode** 是否存在

每个操作都存在一个同步版本和一个非同步版本

触发 **Watch** 的操作：

Znode 被创建

Znode 的数据被修改

Znode 被删除

子节点的列表发生变化（如添加、删除子节点）

watch 只生效一次（想继续用需要重新注册）

通知和重新订阅之间可能发送多个变化

客户端在同一个 **znode** 上，同一种 **watch** 多次注册只会保留一个

新的状态不会随通知传递，需要主动读取

watch 类型

Data watch

监视一个 **znode** 的状态

订阅操作：**get**，**exists**

可能收到的通知：**NodeDataChanged**，**NodeDeleted**，**NodeCreated**

Child watch

订阅操作：**ls/getChildren**

可能收到的通知：**NodeChildrenChanged**

创建/删除 **znode** 时，可能会同时触发：**NodeCreated** 或 **NodeDeleted** + **NodeChildrenChanged**

持久节点：只能通过明确的请求删除

临时节点-会话结束后会自动删除，无法有子节点

持久顺序节点-同持久节点，但是节点名后面会自动附加一个序号（从 00 开始）

元数据

- cZxid: 节点创建时的事务 ID
- mZxid: 表示节点最近一次数据更新时的事务 ID。
- pZxid: 表示节点的子节点列表发生变化时的事务 ID。
- mtime: 上一次被修改的时间
- version: 数据被修改的次数
- cversion: 所有节点被修改的次数
- aclVersion: 权限被更改的次数
- ctime: Znode 创建的时间（以 UNIX 时间戳格式，毫秒为单位）
- dataLength: znode 数据长度
- numChildren: 子节点数量

序列一致性

所有的写入操作都遵循完全的顺序->所有服务器都能以相同的顺序看到修改

写操作处理:

分配唯一事务 id (zxid)

像 quorum 传播写入（向多数节点发送写请求并更新元数据）

发送通知

多节点并发写 不同 znode 时，Zookeeper 保证全局顺序一致

多节点并发写 同一 znode 时可能产生冲突 -> 需要版本控制（CAS）来解决

锁实现

第一页：最基础的锁实现（没有活性保障）

流程

- **Lock()**
 1. 尝试创建节点 `/lock`
 2. 如果创建成功 → 获得锁
 3. 否则：
 - 监听 `/lock` (`watch=true`)
 - 等待 `NodeDeleted` 事件，再重试
- **Unlock()**
删除 `/lock` 节点

问题（活性问题）

- 如果持锁的客户端宕机，但 `/lock` 是持久节点，不会自动删除，其他客户端就永远拿不到锁（死锁）。
- 解决方法：把 `/lock` 改成临时节点（ephemeral），这样持锁客户端断开连接时，节点自动删除，活性得到保障。

第二页：改进成临时顺序节点（但通信量大）

改进点

- 创建锁节点时，不是直接 `/lock`，而是 **ephemeral sequential** 节点，比如 `/lock_0001`。
- 每个客户端都创建一个唯一的临时顺序节点。
- 获取锁的条件：
 - 自己的节点是所有子节点中编号最小的。

流程

1. 创建临时顺序节点 `/lock_xxxx`
2. 获取 `/` 下所有子节点，判断自己是否是最小编号
3. 如果是 → 获得锁
4. 否则 → 监听 `/` 的子节点变化事件（`NodeChildrenChanged`）

问题

- 每次等待都监听 `/` 这个父节点 → 所有客户端都被唤醒（羊群效应）
- 通信量大，因为每次锁释放都会唤醒全部等待者

第三页：进一步优化（只监听前驱节点）

核心思想

- 不监听整个 `/` 节点，而是只监听自己前一个编号的节点（前驱节点）。
- 这样只有下一个候选者会被唤醒，减少无关通信。

流程

1. 创建临时顺序节点 `/lock_xxxx`
2. 获取 `/` 下所有子节点（不设置 `watch`）
3. 如果自己是最小编号 → 获得锁
4. 否则：
 - 找到自己编号的前驱节点
 - `exists(前驱节点, watch=true)`
 - 等到前驱节点被删除，再进入下一轮判断

优点

- 只唤醒真正有资格竞争锁的下一个客户端
- 避免羊群效应
- 通信更少，锁获取延迟更低

服务器类型

leader-主-所有服务器选举出来

follower-从（投票）

observer-观察者（记录决策，但是不参与投票）（可选，但是有用）

生产环境中，服务器数量必须是奇数且 ≥ 3

每个服务器拥有

sid-服务器 id

zxid-事务 id

选举：

比较 zxid

zxid 相同则比较 sid

流程

每轮服务器都会交换自己的投票（sid，zxid）

收到更优的票后，更新自己的票

某个服务器的票获得多数支持->成为 leader

zba 协议

客户端发送写请求给任意服务器

如果是 Follower → 转发给 Leader

Leader 创建提案（proposal），分配 zxid，广播给所有 Follower

Follower 收到提案 → 写入事务日志 → 回复 ACK

Leader 收到多数派 ACK → 广播 commit 消息

所有服务器提交事务 → 更新内存数据 + 通知 Watcher

特点：

必须多数派确认才能提交（保证一致性）

提交是原子性的（要么所有节点都提交，要么不提交）

zxid 保证写操作顺序全局一致

Not Only SQL

SGBD - ACID

Atomicité : une transaction se fait au complet ou pas du tout

Cohérence : l'état du système reste valide à chaque transaction

Isolation : Aucune dépendance possible entre les transactions

Durabilité : une transaction confirmée demeure enregistrée

原子性 (Atomicité): 一个事务要么完全执行, 要么完全不执行

一致性 (Cohérence): 系统在每个事务执行后都保持有效状态

隔离性 (Isolation): 事务之间不能相互依赖

持久性 (Durabilité): 已提交的事务必须被永久保存

关系型数据库的问题: 没办法很好的拓展 Ils ne passent pas à l'échelle

数据异地复制:

好处: 均衡负载

地理位置 (客户端可以联系最近的数据)

鲁棒性

问题: 如何保证强一致性

如何不降低读写操作的延迟

分区容错性如何处理系统因故障/网络延迟而分区的情况?

CAP (只能三选二)

Consistency (一致性 / Cohérence)

所有节点在同一时间看到的数据必须一致。

Tous les nœuds voient les mêmes données au même moment.

Availability (可用性 / Disponibilité)

每个请求都能在有限时间内得到响应 (不保证最新数据)。

Chaque requête reçoit une réponse dans un délai limité (pas forcément les données les plus récentes).

Partition tolerance (分区容错性 / Tolérance au partitionnement)

系统在网络分区 (节点间通信中断) 时仍能继续运行。

Le système continue à fonctionner même en cas de partition réseau.

C&A -> 需要线性化 (每个写入在每个副本上需要以相同顺序被查看)

P&C -> 系统拒绝立即响应写操作的结果

P&A -> 副本数据不一致

BASE

BA: Basically Available 基本上可用

Quelle que soit la charge du SGBD (données/requêtes), on garantie un taux de disponibilité minimal de la donnée.

无论 SGBD 的负载如何（数据/查询），我们都保证数据的最小可用性。

S: Soft-state 软状态

La base NoSQL n'a pas à être fortement cohérente à tout instant

NoSQL 数据库无需在任何时刻都保持强一致性

E: Eventually consistent 最终一致性

À terme, la base atteindra un état cohérent

从长远来看，要达到一个协调一致的状态

NoSQL 数据库常见特性：

查询 API 已简化（无需 SQL）

数据和计算分布式且地理冗余

数据模式非常灵活

数据间无复杂关系（例如：无外键）

适配 BASE 特性

hbase（具有一致性，分区一致性，牺牲了可用性）

非关系型数据库

键值

列

文件

图

normalisation 标准化

每条数据在系统中只有一个拷贝，不重复储存。

避免冗余，更新简单，节省空间

多表 join 性能差

denormalisation 反标准化

在多个表或文档中复制相同的数据

简化加快读取，减少/避免 join

写入时需要保证多处数据一致性

反标准化步骤：

先把数据模型标准化到一个合理程度（避免过度冗余）

分析客户端最常用的查询（找到瓶颈）

在以下场景中可反标准化：

多表数据在关键查询中频繁被关联 →

合并表（Fusion de tables）

增加冗余列（Ajout de colonnes redondantes）

聚合计算开销大 →

在表中直接存储聚合列（Ajout des colonnes d'agrégat dans la table）

分片

将文件片段分布到一组服务器上

目标：

弹性

容错

常见实现方式：

HDFS

分布式 B 树

分布式哈希表

第一页：NoSQL vs. 关系型数据库 (SGBD relationnel) 对比表

维度	NoSQL	关系型数据库 (Relational DB)
数据结构	无固定模式 (schema-less)，列不固定	所有表的结构固定 (schema固定)
数据量	可以处理超大表 (十亿行级别)	典型为中等规模 (百万行级别)
处理方式	偏向分析型、非事务型	强事务处理
存储方式	可按行存储，也可按列存储	通常按行存储
扩展方式	水平扩展 (加机器) + 垂直扩展	垂直扩展 (升级单机)
查询方式	简单 (put/get 操作)	复杂 (SQL, PL/SQL)
空值处理	空值不实际存储	空值存储在数据库中

核心理解

- NoSQL 在设计上追求灵活性和大规模数据处理能力，适合大数据场景。
- 关系型数据库更强调一致性、复杂查询、事务处理，适合结构化和高一致性要求的业务。

选择 NoSQL 时主要考虑：

业务需求：应用是需要强事务，还是更关注大规模读写和灵活性？

基础设施：是否有条件进行分布式部署、是否需要横向扩展？

hBase

特性:

分布式索引系统 -> 负载均衡

基于可靠的分布式文件系统（默认 HDFS）-> 容错能力，数据健壮性

直接对大型数据集进行读写 -> 快速访问

NoSQL 储存:

面向列 -> 适合在线分析处理

关键字 - 值 -> 通过关键字快速访问值

排序 -> 允许按关键字区间检索值

水平与线性拓展 -> 加机器就能成倍增加储存和计算能力

自动数据分区 -> 数据按 **key** 自动划分成不同分区，分到不同节点，分布均衡，无感知

故障自动切换 -> 节点挂了，系统自动把其数据切换到其他节点，高可用性

map-reduce 支持 -> 可以直接通过 map-reduce 进行大规模并行计算

java API -> 与 hadoop 天然集成

集群中，按行分布（每个设备 n 行数据）

RegionServer

WAL

store 每个列族的数据由一个独立的 store 管理

Hmaster

Zookeeper

meta 表：记录了集群中所有 Region 的分布信息以及与这些 Region 相关的元数据。

原子性

一致性

namespace -> table -> 列簇 -> 列 -> (时间戳, 值)(默认保存最新的 3 个)

Namespace1										
Table1	ColumnFamily1				ColumnFamily2					
	Col1		Col2		Col3		Col4		Col5	
Rowkey1	vers2	val1	vers1	val6	vers1	val7	vers3	val10		
							vers2	val11		
	vers1	val2					vers1	val12		
Rowkey2	vers1	val3					vers1	val13	vers1	val16
Rowkey3			vers1	val5	vers1	val9	vers1	val14	vers1	val15

Namespace: 表的逻辑分组

自带两个默认: **hbase** (元数据表)

default (建表时未指定 namespace)

Table: 相当与关系数据库的表

表名是字符串

全称是 **namespace:tablename**

Row: 存放数据的行的唯一 **RowKey** 标识

字节数组

同一个 **RowKey** 所有数据储存在一起

ColumnFamily: 列的逻辑分组

表创建的时候定义 (不可更改)

一行中的所有列族定义相同, 但是可以为空

hbase 中物理储存的最小单位, 同一列族数据会储存在一起

Column: 列族内的细分列, 用列限定符区分

写入数据时动态指定

列名是字节数组

Cell: 储存数据的最小单位

RowKey + ColumnFamily + Column 三元组唯一确定

一个 **Cell** 可以存多个版本

储存的是真正的数据值

Version: **Cell** 在不同时间点的值

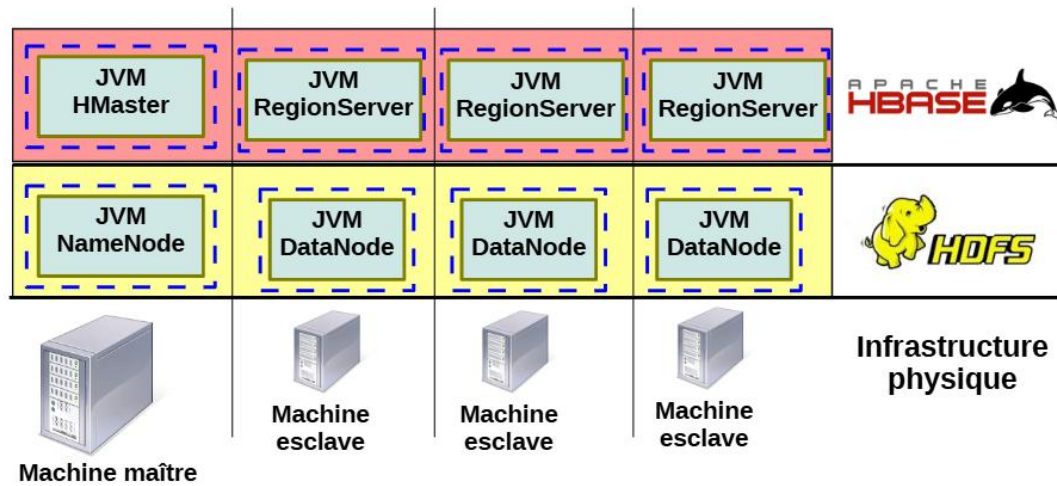
默认用时间戳标记

每个 **cell** 版本数可配置

Value: 最终储存在 **Cell** 中的二进制数据

无类型, 字节数组, **null** 不占储存

全局架构



hbase 与 hdfs 集群一一对应

主节点 = Hmaster

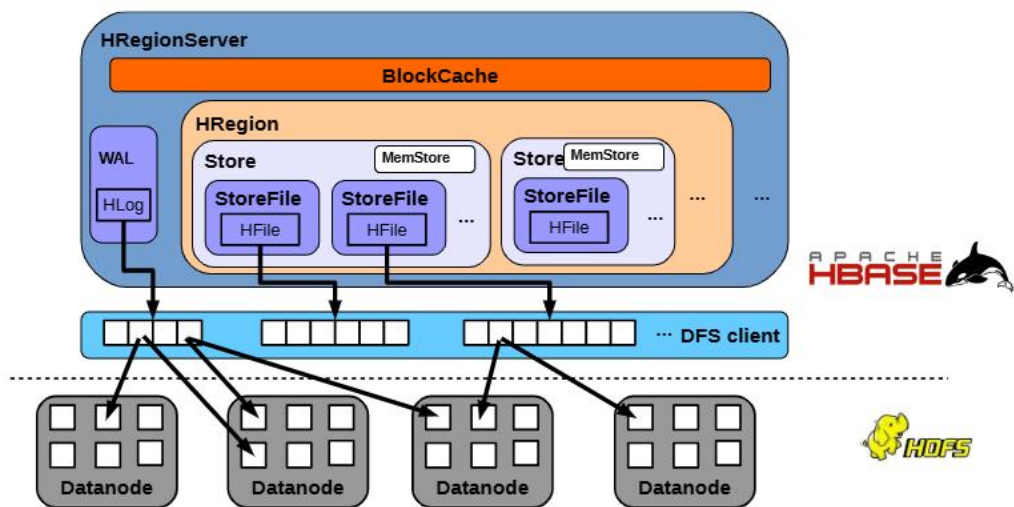
从节点 = ReginServer

集群中数据切分

储存在物理节点上

按 rowkey 排列

通过最小 rowkey 和最大 rowkey 标识



RegionServer: hbase 中的一个服务进程负责一个或多个 HRegion

主要功能:

提供数据操作 API: get, put, delete, next 等。

提供区域操作 API: splitRegion, compactRegion 等。

是访问数据的入口点。

BlockCache: 默认是 LRU (Least Recently Used, 最近最少使用) 缓存。

作用:

缓存数据块 (block) 以加速读取。

所有读操作会先从 BlockCache 里查找, 如果命中直接返回, 减少 HDFS 访问延迟。

WAL (Write Ahead Log, 预写日志):

功能:

在数据写入 MemStore 前, 先将变更记录追加到 WAL。

这样在 RegionServer 宕机时, WAL 可以用于恢复未持久化的数据, 保证数据持久性 (Durability)。

实现:

WAL 存储在 HDFS 上的 HLog 文件中。

HRegion (区域): HBase 表的一个连续行 (rowkey 范围) 的子集。

每个 HRegion 由一个或多个 Store 组成 (每个 ColumnFamily 对应一个 Store)。

一个 RegionServer 可以管理多个 HRegion。

Store（存储单元）：每个 Store 对应一个 ColumnFamily。

组成：

MemStore：内存缓存区，暂存写入的数据。

StoreFile（HFile）：持久化到 HDFS 的文件。

特点：

所有写入先进入 MemStore。

MemStore 满了会触发 flush，将数据写入 HFile。

一个 Store 可以有多个 HFile（通过 compaction 合并）。

表的数据分片结构：

一张表会被切分成多个分区（即 Region）。

每个 Region 由一个 RegionServer 管理。

一个 RegionServer 可以同时管理多个 Region。

一个 Region 里可以有多个 Store。

每个 Store 对应一张表的一个 ColumnFamily。

一个 Store 内部有一个 MemStore（内存写缓存）和多个 StoreFile（HFile 文件）。

每个 StoreFile 对应一个存储文件，存储的是该分区的数据。

Hmaster

协调和监控 RegionServer，某个 rs 挂了，就替换

hbase:meta 表管理，负责表级别和列族的创建，删除，修改等

RegionServer 之间的负载均衡

可以有多个副本，保证高可用

Hmaster 挂了，会有副本补上

故障期间，Hbase 仍然可以工作，因为客户端是通过连接 rs 读写数据的

ZooKeeper

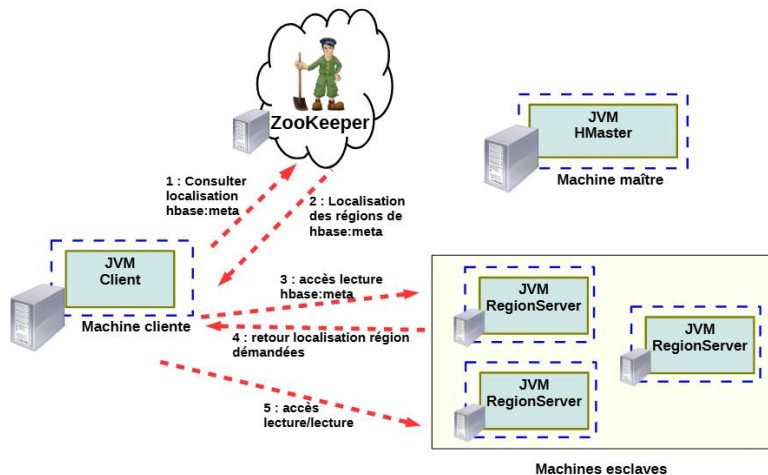
故障检测

元数据定位

hbase:meta 表

维护系统中所有的 region 表

位置储存在 zookeeper 中（只是位置！！！）



客户端查询 hbase:meta 表位置

JVM Client 首先向 ZooKeeper 发送请求，询问 hbase:meta 这张系统表在哪个 RegionServer 上。

ZooKeeper 返回 hbase:meta 所在 RegionServer 的地址

ZooKeeper 查到 hbase:meta 当前在哪台 RegionServer，并把这个位置返回给客户端。

客户端访问 hbase:meta

JVM Client 直接连接到持有 hbase:meta 的 RegionServer，读取 hbase:meta 表内容。

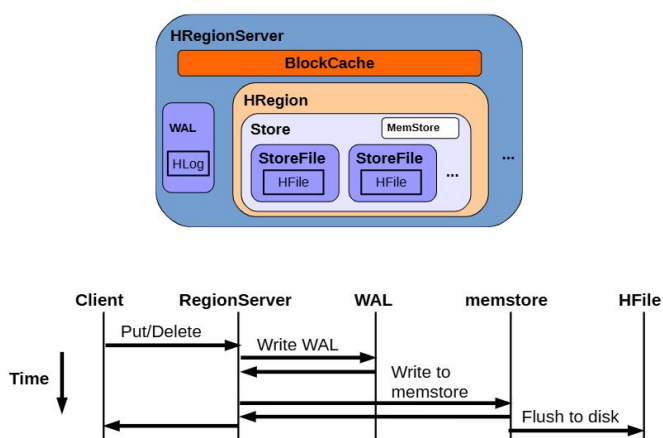
hbase:meta 返回目标 Region 的位置

hbase:meta 表中记录了每个 Region 的元信息，包括：所属表、RowKey 范围、RegionServer 地址等。

客户端从中找到目标数据所在的 RegionServer 地址。

客户端访问目标 RegionServer 读/写数据

客户端直接与对应的 RegionServer 建立连接，执行数据的读取或写入操作（不再经过 ZooKeeper 和 HMaster）。



Mutation:

Put
Delete
Append
Increment

get 单行读取，传入一个 rowkey，返回一个 result（包含所有符合条件的 cell）

scan 多行顺序扫描，传入某行开始/区间，返回一个迭代器

Map-Reduce

执行原理

1. 数据存储在 HBase 的 RegionServer 上（底层是 HDFS 的 HFile）。
2. 当你用 HBase 提供的 TableInputFormat 启动 MapReduce 作业时：
 - Map 阶段的输入数据并不是先从 HBase 全部拉到某个地方再处理，而是直接由 Map 任务向相应的 RegionServer 拉取对应的行数据（Scan 的方式）。
 - Hadoop 会尽量让 Map 任务跑在存有该 Region 数据的节点（数据本地化），减少网络传输。
3. Map 阶段处理完数据后，生成中间结果（写在 HDFS 本地或 shuffle 到 Reduce 节点）。
4. Reduce 阶段完成聚合/计算。
5. 结果写回 HBase（通过 TableOutputFormat），或者写到 HDFS 文件里。
 - 如果写回 HBase，则会通过 RegionServer API（Put/Delete 等 Mutation）把数据放到目标表。

关键点

- MapReduce 作业本身是在 Hadoop/YARN 集群上运行，不是 HBase 内部的一个后台任务。
- HBase 在这个过程中只扮演数据源（Map 输入）和可选的结果存储（Reduce 输出）的角色。
- 数据处理是在 MapReduce 框架的 JVM 里完成的，不是在 RegionServer 里原地算完的。
- 为了性能，MapReduce 会尝试 Map-Task 与 Region 数据节点 co-locate（同机执行），避免全量跨网络搬数据。

Hbase 的三大特性

原子性

一行内的操作是原子的，要么都成功，要么都失败
并发的 mutation 的顺序不会错
跨行不保证原子性

一致性

get 操作返回一个时间点的完整版本
scan 返回的不是整表的一致视图（扫描中有行被更新，就会扫描到新的行）
scan 返回的每一行是一致的

持久性

可见的数据是持久的
提交成功的操作是持久的
提交失败的操作不持久

Cassandra

分布式 NoSQL 数据库

宽列储存型

特性:

P2P 架构 -> 没有中心节点, 任意节点都可以与其他节点互换

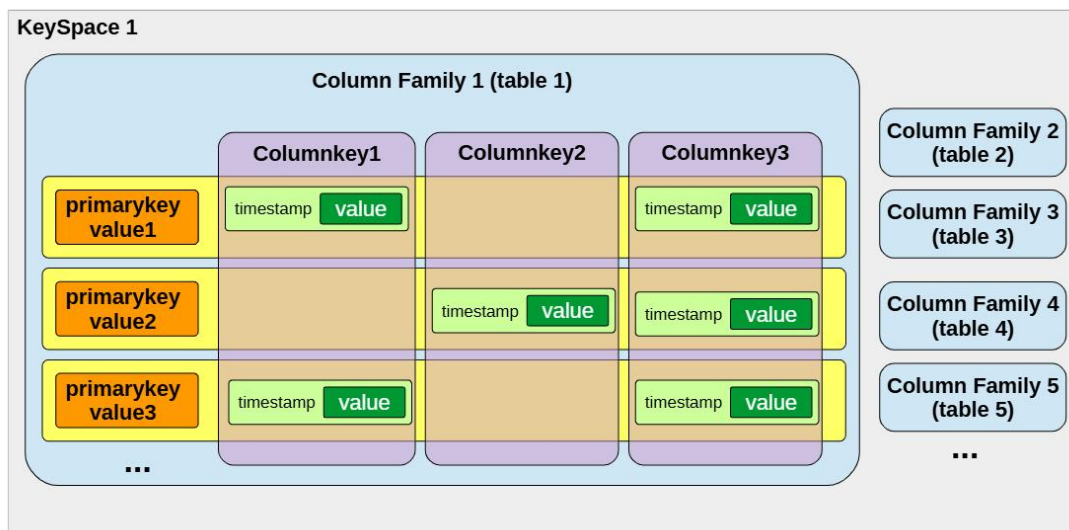
可配置的多数据中心复制机制 -> 容错, 数据永久可用

线性拓展&水平拓展 -> 机器加倍, 储存和计算能力翻倍

可配置副本一致性 -> AP 和 CP 之间权衡的可能性

可与 map-reduce 集成 -> 融入 hadoop

cassandra 查询语言 -> CQL (类似于 SQL 的高级查询语言)



Keyspaces 命名空间: 一组表

RF: 复制因子, 决定每条数据存几份副本

负责策略:

SimpleStrategy: 整个集群用同一个 RF (适合单数据中心开发/测试)。

NetworkTopologyStrategy: 为每个数据中心单独指定 RF (生产常用)。

系统自带 keyspace: system, system_auth, system_distributed, system_schema, system_traces 等

Tables/Column Family 表: 数据行的集合 (类似于关系型数据库的表)

定义主键 (PRIMARY KEY): PRIMARY KEY ((**partition_key**), clustering1, clustering2, ...)

括号内的是 分区键 (决定路由与存放节点)

后面的是 聚簇列 (决定分区内的排序与唯一性)

定义列集合: 每行有哪些列及其类型。

内部属性: 如缓存策略、压实 (compaction) 策略等存储/性能相关设置

Columns 列：列名（键） ↔ 某行上的一个数据项

简单类型：text/string、date、int、float、inet(IP)、boolean、timestamp...

集合类型：list、set、map

tuple：定长元组

选择合适的列类型能直接影响读写与存储效率。

Rows 行/分区

由主键识别：= 分区键（必有） + 若干聚簇列（可选）。

复制粒度：以“分区（行）”为单位在各节点间复制——因此分区键的设计尤为重要（决定数据分布与扩展性）。

行内多列：同一分区内的多条“逻辑记录”按聚簇列排序存放。

Data/Cell 单元格：（某行主键）+（列）+（timestamp） 唯一定位。

timestamp 的用途：

解决跨副本并发写入冲突（最后写入胜出，last write wins）。

保存时间序列（可以一次存多版值）。

副本重同步（repair）时据此比较新旧。

还可以为单元格设置 TTL（存活时间），到期自动过期。

Keyspace -> table(列簇 columnFamily) -> 列 -> （时间戳，值）(默认只保存最新的)

点对点架构

组件：

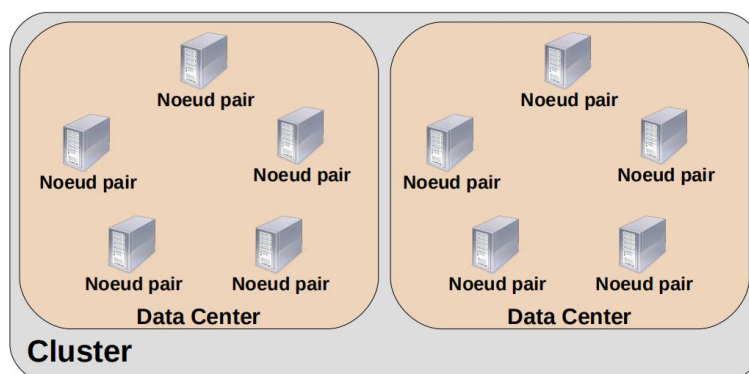
集群 -> 数据中心 -> 机架 -> 节点

节点：存储数据的机器

集群：存储节点的集合

数据中心：存储节点的一个子集

机架：数据中心内节点的一个物理组合



分发/复制基本原则

表拆分的最小粒度是行

行的分布与复制取决于： 哈希函数

副本因子和复制策略

vnode 与物理节点的映射

嗅探器-> 合理分配位置，提高容错能力

分区策略

基于 DHT 进行分区

每行都可能在不同的节点上

每行通过分区键进行哈希，按照范围放入不同的节点中

默认使用 **Murmur3** 哈希算法

新增节点：重新计算每个节点的哈希范围，调整部分数据

失去节点：重新计算每个节点的哈希范围，调整失去节点的原有数据

有序分区

键按字节逐字节排序

数据已排序，区间直接对应键值

分布可能不均匀

删除一个节点都要重新计算每个节点的负责区间

重建环形需要额外的数据移动开销

虚拟节点 vnode

在每个物理节点上划分出更多的虚拟节点

当节点失效时，只需要将该物理节点上的虚拟节点分到其他物理节点上即可，不重新哈希

节点间交流使用 **gossip** 协议，每个节点定期与周围节点交换消息（**redis** 集群同款）

定期发送自身状态+他所知的其他节点状态

优点：没有全局广播，信息逐个邻居传播

定期发送相当于心跳检测

snitch（探测器）

用于决定数据副本如何分布在不同的节点、机架和数据中心中。

基于 gossip

常用 Snitch 策略

Dynamic Snitching（动态 Snitch）：

根据读取延迟的历史记录动态选择最佳副本，提高读取性能。

SimpleSnitch：

适用于单数据中心的简单集群，不考虑拓扑。

RackInferringSnitch：

根据 IP 地址推断数据中心和机架位置。

PropertyFileSnitch：

通过配置文件定义数据中心和机架的拓扑结构。

本地物理储存

Commit log： 记录写入操作

Mem-table： 内存中的缓存表，按键排序

SSTable： 不可变的磁盘文件，保存持久化版本的数据表

持久化

数据先写入 Commit Log

同时写入 Mem-table

Mem-table 满了，进行刷新 flushing，

生成 SSTable，

Mem-table 会被清空，Commit Log 中已持久化的内容会被标注为可被清理

Commit Log 满了之后，会强制关联的 Mem-table 刷盘（写入硬盘），

然后创建新的 Commit Log，然后继续

压缩

定期压缩 SSTable

删除过期/标记为删除的数据

合并重复行

请求流程

客户端联系任意节点

该节点作为协调器，将请求路由到请求对应的节点（数量取决于一致性级别）

协调器汇总返回的数据，返回最新的版本给客户端

如果发现返回的副本不同，则将所有节点都更新成最新版本

一致性级别：定义请求成功完成所需的最小副本数。

常见级别

ONE : 联系最近的有数据的节点，一个返回就行 高可用，低一致

TWO : 俩

THREE : 仨

QUORUM : 半数响应

ALL : 所有副本所在的节点都显示

读操作

客户端发消息给最近节点（协调器）

协调器找到所有相关的节点（哈希过去）

协调器发送请求给节点

节点返回数据

在得到一致性级别的节点回应之后，请求完成

本地读请求

先检查 Mem-table，使用 keys 检查

没有则检查 SSTable，使用布隆过滤器

如果存在于多个 SSTable，则合并后返回最新版本数据

写操作

客户端发消息给最近节点（协调器）

协调器找到所有相关的节点（哈希过去）

协调器发送请求给节点

节点写入，返回消息

得到一致性级别的节点回应之后，请求完成

本地写请求

写入 Commit Log

写入 Mem-table

分布式大数据资源模型

服务必须总是可用的 -> 可扩展 鲁棒
机制:

- 并行和分布式任务
- 分享计算资源并远程储存
- 负载分区
- 按地理分配数据
- 业务复制
- 心跳

依据什么分配分布式系统的角色

p4

网络架构

使用方式 (Usage Applicatif)

唯一 (Unique)：系统仅用于单一应用。

多重 (Multiple)：系统可以托管多个应用程序。

访问模式 (Accès)

私有 (Privé)：仅限单个组织使用。

公共 (Public)：多个组织共享。

混合 (Hybride)：部分资源为私有，部分可共享。

异构性 (Hétérogénéité)

同质 (Homogène)：所有节点具有相同的硬件和软件配置。

异质 (Hétérogène)：节点的硬件或软件配置不同。

可控性 (Contrôlabilité)

可控 (Contrôlé)：节点由管理员或专用控制节点管理。

不可控 (Non contrôlé)：节点完全自主，不受外部管理。

动态性 (Dynamicité)

低动态 (Faible)：节点数量固定，只有在故障时才变化。

高动态 (Forte)：节点数量随时间变化，可能因环境波动、加入或离开节点等因素改变。

地理位置 (Localité Géographique)

集中式 (Centralisé)：所有节点位于相同的地理位置。

分布式 (Délocalisé)：节点分布在不同的地理位置。

可靠性 (Fiabilité)

可靠 (Fiable)：节点仅可能发生宕机 (pannes franches)，并且执行程序时是可信的。

不可靠 (Non fiable/Byzantin)：某些节点可能被恶意攻击或损坏，并可能返回错误的计算结果。

网络连接 (Nature des Liens Réseau)

局域网 (Réseau Local)：节点通过局域网通信。

互联网 (Internet)：节点通过互联网通信。

移动自组织网络 (Mobile Ad hoc Network, MANet)：节点通过无线电直接进行点对点通信。

集群计算 Cluster computing: Agréger de la puissance de calcul localement
汇聚本地计算能力

原理:

整合本地计算能力 $\Rightarrow \Rightarrow$ 高性能网络
构建超级计算机
提供高可用性服务

适合:

Calcul haute performance 高性能计算平台
Service hautement disponible 高可用服务

缺点:

昂贵
资源有限而且静态
大部分时间不能充分利用所有资源

Apparition 显现	Années 80 80 年代
Usage applicatif 使用应用场景	unique ou multiple 唯一或多个
Accès 访问	privé 私人
Hétérogénéité 异构性	très faible 非常低
Contrôlabilité 可控性	totale 完全
Dynamicté 动态性	très faible 非常低
Localité géographique 地理位置	centralisé 集中的
Fiabilité 可靠性	forte 强
Nature des liens réseau 网络连接类型	réseau local 局域网

对等计算 Peer-to-peer computing: Collaboration de nœuds sans notion de hiérarchie
没有层次概念的节点协作

原理:

无层级节点协作 $\Rightarrow \Rightarrow$ 无中心化管理, 更易于扩展

降低维护成本 (每个节点自我管理)

节点可自主选择:

在系统中的参与程度

连接或断开系统

可引入匿名机制

节点去中心化

适合:

Stockage distribué 分布式储存

Partage de données 数据共享

缺点:

效率低

冗余成本高 Coût de redondance élevé

计算性能差

Apparition 显现	Fin des années 90 90 年代末
Usage applicatif 使用应用场景	unique 独特
Accès 访问	public 公共
Hétérogénéité 异构性	moyenne 平均
Contrôlabilité 可控性	individuelle 个性化
Dynamicté 动态性	moyenne à forte 中等至强
Localité géographique 地理位置	délocalisé 外包
Fiabilité 可靠性	faible 脆弱
Nature des liens réseau 网络连接类型	Internet dans beaucoup de cas 在许多情况下, 互联网

移动计算 Mobile computing: Système composé de nœuds autonomes et mobiles

由自主移动节点组成的系统

原理:

由自主移动节点组成的系统

可实现物理直接通信 (例如: 无线电波)

可在任何演化环境中连接

网络链接和基础设施非常动态

适合:

Applications embarquées et réseaux mobiles 嵌入式和移动网络

Véhicules autonomes 自动驾驶车辆

Réseau de robots, de capteurs 机器人网络, 传感器网络

缺点:

网络不稳定

低带宽 Faible bande passante

受能源限制

Apparition 显现	années 90 90 年代
Usage applicatif 使用应用场景	multiple 多个
Accès 访问	privé 私人
Hétérogénéité 异构性	moyenne à forte 平均到强
Contrôlabilité 可控性	locale à un device 本地到一个设备
Dynamicté 动态性	très forte 非常强
Localité géographique 地理位置	délocalisé 外包
Fiabilité 可靠性	faible 脆弱的
Nature des liens réseau 网络连接类型	mobile, internet 移动, 互联网

网格计算（Grid computing）

Partage ou mise en commun de ressources parmi plusieurs institutions

在多个机构之间共享或汇集资源

原理：

多个机构之间的资源共享或共享

分布式资源 ⇒ 降低运营成本

跨地域集群/机器联邦

适合：

Calcul parallèle scientifique 科学并行计算

Service hautement disponible 高度可用的服务

缺点

成本高

需要好的网络连接

有一个共同的管理策略

Apparition 显现	Fin des années 90 90 年代末
Usage applicatif 使用应用场景	multiple 多个
Accès 访问	privé ou hybride 私有或混合
Hétérogénéité 异构性	faible 低
Contrôlabilité 可控性	locale à un site locale 到一个网站
Dynamicté 动态性	faible 低
Localité géographique 地理位置	délocalisé 外包
Fiabilité 可靠性	forte 强大
Nature des liens réseau 网络连接类型	réseau local et inter très haut débit 局域网和超高速互联网

志愿者计算 Volunteer computing:

Même principe que le Grid Computing mais avec des ordinateurs personnels

原理与网格计算相同，但使用的是个人电脑

原理:

与网格计算相同原理，但使用个人电脑 $\Rightarrow \Rightarrow$ 运营成本极低

节点可按其所有者的意愿随时可用（例如：在待机状态下）

节点所有者可获得相应报酬

适合:

Analyse de données de capteurs pour recherche scientifique 传感器数据分析用于科学研究

Domotique, radiateur numérique 智能家居，数字散热器

缺点:

不稳定

需要划分任务

Apparition 显现	Fin des années 90 90 年代末
Usage applicatif 使用应用场景	unique ou multiple 独特或多个
Accès 访问	public 公共
Hétérogénéité 异构性	très forte 非常强大
Contrôlabilité 可控性	individuelle 个性化的
Dynamicté 动态性	très forte 非常强大
Localité géographique 地理位置	délocalisé 外包
Fiabilité 可靠性	très faible 非常低
Nature des liens réseau 网络连接类型	Internet 互联网

云计算 Cloud computing

Tout devient service → Everything-as-a-Service (XaaS)

万物皆服务 → 一切即服务 (XaaS)

IaaS: Infrastructure as a Service (基础设施即服务)

PaaS: 平台即服务

SaaS: 软件即服务

原理:

万物皆服务 → 一切即服务 (XaaS)

通过虚拟化实现无限资源

供应商与消费者之间的服务质量合同 (服务水平协议)

按需计算: "只付所耗" ⇒ 极大的灵活性和资源利用率的提升

Un XaaS de niveau N :

- peut utiliser plusieurs XaaS de niv. $N - 1$
- peut fournir plusieurs XaaS de niv. $N + 1$
- peut être fédéré avec un XaaS de niveau N
⇒ "Sky computing"

向下依赖: 一个高层服务 (如 SaaS) 会用到多个低层服务 (如 PaaS)。

向上输出: 一个底层服务 (如 IaaS) 可以支持多个更高层服务 (如 PaaS)。

同级合作: 同一层的服务可以互相连接, 形成跨平台、跨云的 "Sky Computing"。

弹性

Capacité d'un système à adapter dynamiquement sa configuration au cours de son exécution en fonction de sa charge.

系统在运行过程中根据负载动态调整其配置的能力。

基础设施的弹性: 系统能够根据负载变化动态调整资源分配

分配 allouer

负载 charge

水平弹性 (Elasticité Horizontale)

通过增加或减少节点数量 (服务器、虚拟机、容器等) 来适应负载变化。

垂直弹性 (Elasticité Verticale)

通过增加或减少单个节点 (服务器、虚拟机、容器等) 的计算资源 (CPU、RAM、存储等) 来适应负载变化。

Élasticité sur l' infrastructure (基础设施层的弹性)

Augmenter CPU/RAM: 增加 CPU / 内存 (垂直扩展, Elasticité Verticale)

Diminuer CPU/RAM: 减少 CPU / 内存 (垂直收缩)

Ajout VM: 增加虚拟机 (水平扩展, Elasticité Horizontale)

Suppression VM: 删除虚拟机 (水平收缩)

垂直弹性 (Vertical Scaling): 单台机器变强或变弱 (加/减 CPU、内存)。

水平弹性 (Horizontal Scaling): 机器数量增加或减少 (加/删 VM)。

Élasticité logicielle (软件层的弹性)

Améliorer Fonctionnalité: 提升功能 (垂直扩展)

Dégrader fonctionnalité: 降低功能 (垂直收缩)

Ajout Fonctionnalité: 增加功能 (水平扩展)

Suppression Fonctionnalité: 删除功能 (水平收缩)

垂直弹性: 同一个软件增加/减少功能复杂度, 比如 Gmail 增加地图、云盘支持。

水平弹性: 增加/减少同类功能模块的数量, 比如更多的邮件服务、更多的 API 节点。

适合:

具有强烈的 QoS 概念的在线服务 (Qos, 服务质量)

数据存储

缺点:

Trouver une configuration optimale du système 寻找系统最佳配置

Quand doit-on reconfigurer? 何时需要重新配置?

Comment conhammer le moins? 如何尽量减少?

Comment respecter les SLA des clients? 如何遵守客户的服务水平协议 (SLA)?

Outils d'exploitation complexes 复杂的运营工具

Apparition 显现	Fin des années 2000 2000 年代末期
Usage applicatif 使用应用场景	multiple 多个
Accès 访问	public, privé ou hybride 公共的、私有的或混合的
Hétérogénéité 异构性	faible 低
Contrôlabilité 可控性	pleine (fournisseur et consommateur) 完全的 (供应商和消费者)
Dynamicité 动态性	matérielles : faible, virtuelles : forte 实体: 低, 虚拟: 高
Localité géographique 地理位置	matérielles : centralisé ou délocalisé, virtuelles : ? 设备: 集中式或外包式, 虚拟式: ?
Fiabilité 可靠性	moyenne à forte 平均到强
Nature des liens réseau 网络连接类型	réseau dédié ou Internet 专用网络或互联网

边缘计算 Edge computing:

在靠近用户的位置进行计算和储存

原理:

Privilégier le traitement et le stockage en bordure de réseau 优先考虑网络边缘的处理和存储

Placement le plus proche possible du client final 尽可能靠近最终客户

limiter l'utilisation du réseau 限制网络的使用

limiter la dépendance avec les noeuds du cœur de réseau 限制对网络核心节点的依赖

améliorer le temps de réponse du système 提高系统的响应时间

适合:

Objets connectés, domotique 互联物体、家庭自动化

Applications nécessitant une grande autonomie et une faible latence

需要长时间自主运行和低延迟的应用

缺点:

管理分布式数据

服务放置 Placement des services

安全

Apparition 显现	années 2010 2010 年代
Usage applicatif 使用应用场景	multiple 多个
Accès 访问	public, privé ou hybride 公共的、私有的或混合的
Hétérogénéité 异构性	forte 强大
Contrôlabilité 可控性	partielle 部分
Dynamicté 动态性	faible coté cœur de réseau, forte coté IoT 网络核心方面较弱，物联网方面较强
Localité géographique 地理位置	délocalisé 外包
Fiabilité 可靠性	faible à moyenne 低至中等
Nature des liens réseau 网络连接类型	réseau dédié, mobile, internet 专用网络、移动网络、互联网

雾计算

Union des principes du Cloud Computing et du Edge Computing

云计算和边缘计算的结合

原理:

云计算与边缘计算原理的联合

普适计算

适合:

Service avec forte notion de QoS 具有强大 QoS 概念的服务

Objets connectés 联网对象

Application nécessitant une grande autonomie et une faible latence

要求长自主性和低延迟的应用

Réseaux et applications mobiles type 5G

5G 型移动网络和应用

缺点:

系统规模大, 复杂

需要控制, 部署, 监控, 安全

contrôler, déployer, monitorer, et à sécuriser

Apparition 显现	fin des années 2010 2010 年代末期
Usage applicatif 使用应用场景	multiple 多个
Accès 访问	public, privé ou hybride 公共的、私有的或混合的
Hétérogénéité 异构性	très forte 非常强大
Contrôlabilité 可控性	partielle 部分
Dynamicté 动态性	très forte 非常强大
Localité géographique 地理位置	délocalisé 外包
Fiabilité 可靠性	faible à moyenne 较低到中等
Nature des liens réseau 网络连接类型	tout type de réseau 所有类型的网络